

■ Basic Information

Experiment No: 02

Date: 11/09/2026

Title: UPI Wallet Payment System

■ Objective / Aim

To design and implement a UPI Wallet Payment System in Java using interfaces, classes, encapsulation, custom exceptions, and exception handling, simulating real-world digital payment operations such as wallet creation, money addition, UPI payments, balance checking, and transaction validation.

■■ Problem Statement

Design a digital wallet system for a user. The system should allow a user to:

- Create a wallet with a name, mobile number, and UPI ID.
- Add money to the wallet.
- Make a payment using the UPI ID of another user.
- Check the available wallet balance.
- Validate the UPI ID before processing a payment.
- Prevent payments when the balance is insufficient.
- Prevent invalid or negative transaction amounts.
- Handle invalid transactions using appropriate exceptions.
- Display the final wallet details and transaction status.

The program must demonstrate:

- Interface
- Classes and Objects
- Encapsulation
- String operations
- Custom Exceptions
- try-catch-finally exception handling

■ Class and Interface Design

1. PaymentService Interface

Define the common operations:

```
pay()
checkBalance()
```

2. Wallet Class

Store:

- User name
- Mobile number
- UPI ID
- Balance

The data members should be declared private to demonstrate encapsulation.

Methods:

- addMoney()
- getBalance()
- displayWalletDetails()

3. UPIPayment Class

Implement the PaymentService interface.

Methods:

- pay()
- checkBalance()

4. Custom Exceptions

Create:

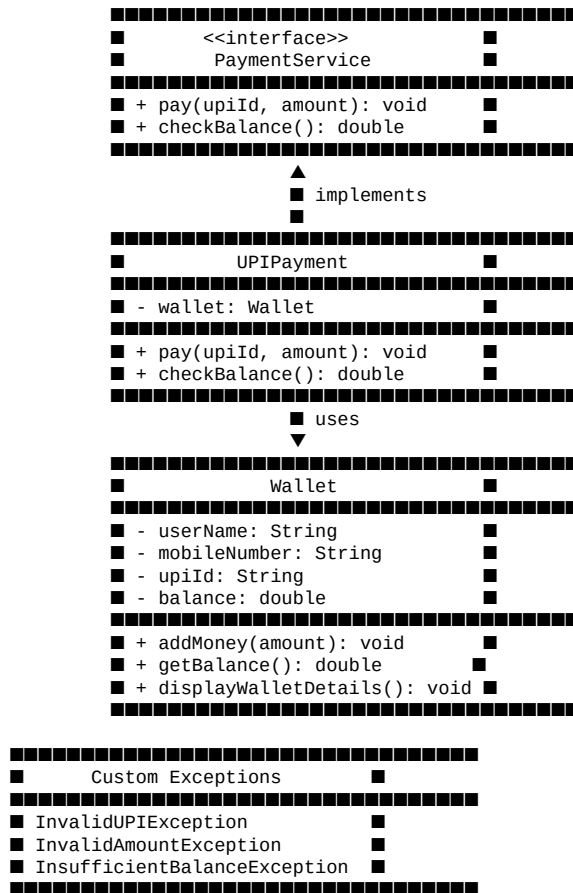
- InsufficientBalanceException
- InvalidUPIException
- InvalidAmountException

■ Steps to do the task

1. Create an interface named PaymentService.
2. Declare pay() and checkBalance() methods in the interface.
3. Create a Wallet class.
4. Declare user information and balance as private data members.
5. Create a constructor to initialize the wallet details.
6. Define addMoney() to add money to the wallet.
7. Create an UPIPayment class that implements PaymentService.
8. Create an object of Wallet and associate it with the UPIPayment object.
9. Validate the UPI ID using String operations.

10. If the UPI ID is invalid, throw InvalidUPIException.
11. Validate the payment amount.
12. If the amount is zero or negative, throw InvalidAmountException.
13. Check whether sufficient balance is available.
14. If the balance is insufficient, throw InsufficientBalanceException.
15. If all validations are successful, deduct the payment amount from the wallet balance.
16. Display the payment success message.
17. Use try-catch-finally to handle transaction exceptions.

■ UML Class Diagram



■ Source Code

```

interface PaymentService {
    void pay(String upiId, double amount)
        throws InvalidUPIException, InvalidAmountException,
            InsufficientBalanceException;

    double checkBalance();
}

class InvalidUPIException extends Exception {
    public InvalidUPIException(String message) {
        super(message);
    }
}

class InvalidAmountException extends Exception {
    public InvalidAmountException(String message) {
        super(message);
    }
}

class InsufficientBalanceException extends Exception {
    public InsufficientBalanceException(String message) {
        super(message);
    }
}

class Wallet {
    private String userName;
    private String mobileNumber;
    private String upiId;
    private double balance;

    public Wallet(String userName, String mobileNumber,
        String upiId, double balance) {
        this.userName = userName;
        this.mobileNumber = mobileNumber;
    }
}
  
```

```

        this.upiId = upiId;
        this.balance = balance;
    }

    public void addMoney(double amount) throws InvalidAmountException {
        if (amount <= 0) {
            throw new InvalidAmountException(
                "Amount must be greater than zero.");
        }
        balance += amount;
        System.out.println("Money added: Rs." + amount);
    }

    public double getBalance() {
        return balance;
    }

    public void deductMoney(double amount) {
        balance -= amount;
    }

    public void displayWalletDetails() {
        System.out.println("User Name      : " + userName);
        System.out.println("Mobile Number : " + mobileNumber);
        System.out.println("UPI ID       : " + upiId);
        System.out.printf("Balance      : Rs.%.2f%n", balance);
    }
}

class UPIPayment implements PaymentService {
    private Wallet wallet;

    public UPIPayment(Wallet wallet) {
        this.wallet = wallet;
    }

    @Override
    public void pay(String upiId, double amount)
        throws InvalidUPIException, InvalidAmountException,
        InsufficientBalanceException {

        if (upiId == null || !upiId.contains("@")) {
            throw new InvalidUPIException("Invalid UPI ID.");
        }

        if (amount <= 0) {
            throw new InvalidAmountException(
                "Payment amount must be greater than zero.");
        }

        if (amount > wallet.getBalance()) {
            throw new InsufficientBalanceException(
                "Insufficient wallet balance.");
        }

        wallet.deductMoney(amount);
        System.out.println("Payment successful to " + upiId);
        System.out.println("Amount paid: Rs." + amount);
    }

    @Override
    public double checkBalance() {
        return wallet.getBalance();
    }
}

public class UPIWalletDemo {
    public static void main(String[] args) {

        Wallet wallet = new Wallet(
            "Ananya", "9876543210", "ananya@upi", 5000);

        UPIPayment payment = new UPIPayment(wallet);

        System.out.println("===== WALLET DETAILS =====");
        wallet.displayWalletDetails();

        try {
            System.out.println("\nAdding Rs.2000...");
            wallet.addMoney(2000);

            System.out.println("\nMaking UPI payment...");
            payment.pay("rahul@upi", 2500);

        } catch (InvalidUPIException |
            InvalidAmountException |
            InsufficientBalanceException e) {

            System.out.println("Transaction failed: "
                + e.getMessage());
        } finally {

```

```
        System.out.println("\n===== FINAL WALLET DETAILS =====");
        wallet.displayWalletDetails();
        System.out.printf("Available Balance: Rs.%.2f%n",
            payment.checkBalance());
    }
}
```

■ Compilation & Execution Commands

```
javac UPIWalletDemo.java
java UPIWalletDemo
```

■■ Output Screen

```
===== WALLET DETAILS =====
User Name      : Ananya
Mobile Number  : 9876543210
UPI ID         : ananya@upi
Balance       : Rs.5000.00

Adding Rs.2000...
Money added: Rs.2000.0

Making UPI payment...
Payment successful to rahul@upi
Amount paid: Rs.2500.0

===== FINAL WALLET DETAILS =====
User Name      : Ananya
Mobile Number  : 9876543210
UPI ID         : ananya@upi
Balance       : Rs.4500.00
Available Balance: Rs.4500.00
```

■ Result

The UPI Wallet Payment System was successfully implemented in Java using an interface, classes, encapsulation, custom exceptions, String validation, and try-catch-finally exception handling. The program demonstrates wallet creation, money addition, UPI payment processing, balance checking, and transaction validation.